

Guía de ejercicios - JWT



¡Hola! Te damos la bienvenida a esta nueva guía de estudio.

¿En qué consiste esta guía?

Esta guía tiene como objetivo explicar cómo se implementa la autenticación y autorización con JWT en una aplicación de React.

¡Vamos con todo!



Tabla de contenidos

Guía de ejercicios - JWT	1
¿En qué consiste esta guía?	1
Tabla de contenidos	1
¿Qué es la autenticación y autorización?	2
¿Qué es JWT?	2
¿Cómo funciona JWT?	2
¡Manos a la obra! - Implementación de JWT en una aplicación de React	3
Explicación paso a paso del código	4
Utilizando el token JWT	4
Explicación paso a paso del código	5
Custom Hooks	6
Conclusión	8



¡Comencemos!

¿Qué es la autenticación y autorización?

La autenticación es el proceso de verificar la identidad de un usuario, mientras que la autorización es el proceso de determinar si un usuario tiene permiso para acceder a un recurso específico.

¿Qué es JWT?

JWT (JSON Web Token) es un estándar abierto basado en JSON que define una forma compacta y autónoma para transmitir información de forma segura entre dos partes como un objeto JSON. Esta información puede ser verificada y confiable porque está firmada digitalmente. JWT se utiliza para autenticar usuarios y autorizarlos a acceder a recursos específicos.

Si visitas: jwt.io, puedes ver un token JWT y su contenido.

Un token JWT consta de tres partes separadas por un punto (.):

1. **Encabezado (Header):** Contiene el tipo de token y el algoritmo de firma.
2. **Cuerpo (Payload):** Contiene la información del usuario y los datos adicionales.
3. **Firma (Signature):** Contiene la firma digital que se utiliza para verificar la autenticidad del token.

Un token JWT

JavaScript

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

¿Cómo funciona JWT?

1. El cliente envía un nombre de usuario y contraseña al servidor.
2. El servidor verifica las credenciales y genera un token JWT.
3. El servidor envía el token JWT al cliente.
4. El cliente almacena el token JWT y lo envía en el encabezado de la solicitud HTTP.
5. El servidor verifica el token JWT y autoriza al cliente a acceder a los recursos.



¡Manos a la obra! - Implementación de JWT en una aplicación de React

- **Paso 1:** Descarga el “Material de apoyo - Implementación de JWT en una aplicación de React”.
- **Paso 2:** Instala las dependencias con: `npm install` y ejecuta el modo desarrollo con `npm run dev`.
- **Paso 3:** Vamos a generar el siguiente componente Login que se encargará de hacer login en nuestra aplicación. Antes de comenzar, verifica que tienes levantado el servidor (backend) que estás utilizando en el desarrollo de tus hitos.

El endpoint `/api/auth/login` recibe un email y una contraseña y devuelve un token JWT.

```
JavaScript
import { useState } from "react";

const Login = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    const response = await fetch("http://localhost:5000/api/auth/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        email,
        password,
      }),
    });

    const data = await response.json();
    alert(data?.error || "Authentication successful!");
    localStorage.setItem("token", data.token);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        placeholder="Email"
      />
    </form>
  );
};
```

```
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Login</button>
    </form>
  );
};

export default Login;
```

- **Paso 4:** Para probar, utiliza las siguientes credenciales:

```
JavaScript
{
  "email": "test@test.com",
  "password": "123123"
}
```

Explicación paso a paso del código

1. Importamos las dependencias necesarias y definimos el componente Login.
2. Creamos dos estados email y password para almacenar los valores de los campos de entrada.
3. Creamos una función handleSubmit que se ejecuta cuando se envía el formulario.
4. Hacemos una petición POST a la ruta /api/auth/login con el email y la contraseña.
5. Almacenamos el token JWT en el localStorage.
6. Creamos un formulario con dos campos de entrada para el email y la contraseña.
7. Al hacer clic en el botón Login, se ejecuta la función handleSubmit.

En este ejemplo estamos persistiendo el token JWT en el localStorage, así podremos acceder a él en cualquier parte de la aplicación.

Utilizando el token JWT

- **Paso 5:** Una vez que hemos obtenido el token JWT, podemos utilizarlo para acceder a rutas protegidas en nuestra aplicación. Vamos a crear un componente Profile que se encargará de mostrar el perfil del usuario autenticado. src\pages\Profile.jsx

JavaScript

```
import { useEffect, useState } from "react";

const Profile = () => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    const token = localStorage.getItem("token");

    if (token) {
      fetch("http://localhost:5000/api/auth/me", {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      })
        .then((response) => response.json())
        .then((data) => setUser(data));
    }
  }, []);

  return (
    <div>
      {user ? (
        <p>Email: {user.email}</p>
      ) : (
        <p>Please login to view your profile.</p>
      )}
    </div>
  );
};

export default Profile;
```

Explicación paso a paso del código

1. Importamos las dependencias necesarias y definimos el componente Profile.
2. Creamos un estado user para almacenar la información del usuario.
3. Utilizamos el hook useEffect para hacer una petición a la ruta /api/auth/me al cargar el componente.
4. Obtenemos el token JWT del localStorage.
5. Hacemos una petición GET a la ruta /api/auth/me con el token JWT en el encabezado.
6. Almacenamos la información del usuario en el estado user.

7. Mostramos el email del usuario si está autenticado, de lo contrario mostramos un mensaje.

Cómo puedes ver en el código anterior, estamos enviando el token JWT en el encabezado de la solicitud HTTP. De esta manera, el servidor puede verificar la autenticidad del token y autorizar al usuario a acceder a la ruta `/api/auth/me`.

Así es como puedes implementar la autenticación y autorización con JWT en una aplicación de React. Puedes personalizarlo y adaptarlo a tus necesidades. Recuerda que en secciones anteriores se explicó como hacer rutas protegidas en React.

Custom Hooks

Los Custom Hooks son una característica de React que nos permite extraer lógica de componentes en funciones reutilizables.

En el proyecto de apoyo se encuentra un Custom Hook llamado ``useInput`` que nos permite manejar el estado de un campo de entrada en un formulario, además de un ejemplo de cómo utilizarlo en ``RegisterWithCustomHooks.jsx``.

- **Paso 6:** Un ejemplo de Custom Hook sería el siguiente: `src\hooks\useInput.jsx`

```
JavaScript
import { useState } from "react";

const useInput = (initialValue) => {
  const [value, setValue] = useState(initialValue);

  const handleChange = (e) => {
    setValue(e.target.value);
  };

  return {
    value,
    onChange: handleChange,
  };
};

export default useInput;
```

- ❖ En este ejemplo, hemos creado un Custom Hook llamado `useInput` que nos permite manejar el estado de un campo de entrada en un formulario.
- ❖ El Custom Hook `useInput` recibe un valor inicial y devuelve un objeto con dos propiedades: `value` y `onChange`.

- ❖ value es el valor actual del campo de entrada y onChange es una función que se ejecuta cuando el campo de entrada cambia.
- **Paso 7:** Para utilizar este Custom Hook en un componente, simplemente importamos la función y la llamamos:

```
JavaScript
import useInput from "../hooks/useInput";

const RegisterWithCustomHooks = () => {
  const email = useInput("");
  const password = useInput("");

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(email.value, password.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        placeholder="Email"
        {...email}
      />
      <input
        type="password"
        placeholder="Password"
        {...password}
      />
      <button type="submit">Submit</button>
    </form>
  );
};

export default RegisterWithCustomHooks;
```

De esta manera, podemos reutilizar la lógica de manejo de estado en múltiples componentes sin tener que repetir el código. En este caso, llamamos a useInput dos veces para manejar el estado de dos campos de entrada en un formulario. Luego utilizamos las propiedades value y onChange en los campos de entrada.

- ❖ {...email} es una forma abreviada de escribir value={email.value} onChange={email.onChange}.
- ❖ {...password} es una forma abreviada de escribir value={password.value} onChange={password.onChange}.

Esto se conoce como "destructuring" en JavaScript y nos permite pasar múltiples propiedades a un componente de forma más concisa.

Los Custom Hooks son una herramienta poderosa que nos permite escribir código más limpio y modular en nuestras aplicaciones de React.

Conclusión

- En esta guía hemos aprendido cómo implementar la autenticación y autorización con JWT en una aplicación de React. Hemos visto cómo hacer login y obtener el perfil del usuario autenticado utilizando tokens JWT.
- Además hemos conocido los Custom Hooks en React, una característica que nos permite extraer lógica de componentes en funciones reutilizables.
- Espero que esta guía te haya sido útil y te haya ayudado a comprender mejor cómo funciona la autenticación y autorización en una aplicación de React.